Class Summary: SqLiteGraphRepository

The SqLiteGraphRepository class is a fundamental component of the Instancium Lens infrastructure layer (InstanciumLens.Infrastructure) [cite: 1]. It is responsible for persisting and retrieving the application's knowledge graph to and from a local SQLite database [cite: 1]. This implementation fulfills the contract defined by the IGraphRepository interface located in the core domain layer (InstanciumLens.Core) [cite: 1].

Core Responsibilities and Design

The primary role of this class is to handle the Pass 3: Graph Building phase of the document analysis pipeline [cite: 1]. It translates the in-memory Knowledge Graph object (containing GraphEntity and GraphRelationship models) into a relational database structure and vice versa [cite: 1].

Key Architectural Features

- **Interface Implementation:** It implements IGraphRepository, adhering to the Dependency Inversion principle [cite: 1]. The core application logic interacts with this interface, remaining agnostic of the underlying SQLite implementation [cite: 1].
- **Targeted Storage Strategy & Multi-Graph Support:** Instead of a centralized application database, it stores the SQLite databases within a hidden subfolder named .instancium located directly inside the user-selected folder containing the analyzed documents. To support multiple domains (e.g., legal vs. engineering), the database name is now dynamically generated based on the selected profile (e.g., knowledge_graph_legal.db).
- **Portability and Network Readiness:** By co-locating the database with the source documents, the knowledge graph becomes portable [cite: 1]. Users can move the folder or place it on a network drive, and the indexed data travels with it, eliminating the need to re-run the computationally expensive Pass 2 (Entity Resolution) [cite: 1].

Initialization (Constructor)

The constructor SqLiteGraphRepository(string targetFolderPath, string profileName = "default") is responsible for setting up the environment dynamically based on user input and the AI-generated schema profile.

1. **Path Validation:** It ensures the provided targetFolderPath is not null or empty [cite: 1].
2. **Hidden Folder Creation:** It constructs the path for the hidden .instancium folder. If this directory does not exist, it creates it and applies the FileAttributes.Hidden attribute using

Windows file system APIs to ensure the technical files remain invisible to the non-technical user [cite: 1].

3. **Database Initialization:** It sanitizes the provided profileName and constructs the SQLite connection string pointing to the dynamically named .db file within the hidden folder, then calls the private InitializeDatabase() method.

Database Schema Initialization

The private InitializeDatabase() method ensures that the necessary tables exist upon instantiation [cite: 1]. It connects to the SQLite database and executes SQL to create two primary tables mapping to the domain models [cite: 1]:

- **Entities Table:** Stores nodes (Id, Name, Type, Description) [cite: 1].
- **Relationships Table:** Stores edges (SourceEntityId, TargetEntityId, RelationType, Description, Weight) [cite: 1]. A composite primary key is used to prevent duplicate relationships [cite: 1].

Core Methods

Method Name	Description & Mechanism
SaveGraphSegmentAsync	Implements the contract to save a segment of the knowledge graph into the database during the Pass 3 phase [cite: 1]. • **Transaction Management:** Wraps the insertion operations in a database transaction (BeginTransaction()) to ensure data integrity; if an error occurs, the entire segment is rolled back [cite: 1]. • **Idempotent Operations:** Utilizes the SQL INSERT OR REPLACE INTO command [cite: 1]. This ensures safe updates without primary key constraint errors, effectively merging new knowledge with the existing

Method Name	Description & Mechanism
	graph [cite: 1].
LoadFullGraphAsync	Retrieves the entire stored knowledge graph from the SQLite database and reconstructs it into a KnowledgeGraph object in RAM [cite: 1]. ● **Data Mapping:** Executes SELECT queries against both the Entities and Relationships tables, instantiates GraphEntity and GraphRelationship objects respectively, and populates the returned KnowledgeGraph [cite: 1].
GetRelevantSubGraphAsync	Actively performs a highly optimized search to extract only the relevant subgraph based on user intent, overriding the previous NotImplementedException [cite: 1]. ● **Algorithm:** Utilizes safe, parameterized SQL LIKE queries to find nodes that match the extracted keywords. Using a smart SQL JOIN, it instantly pulls all connecting relationships (where the matched node is a Source or Target) and fetches the neighboring nodes. ● **Memory Efficiency:** Ensures the local AI model receives a lightweight, focused context instead of loading gigabytes of graph data into memory, solving memory fragmentation issues and enabling ultra-fast query resolution.